

# GDA n8n Response Standard

Date: April 27, 2026

## Purpose

React should not have to guess what an n8n workflow meant. Every high-value GDA API workflow should return a predictable JSON envelope so the dashboard, QA Center, and future automation can tell the difference between healthy data, empty data, unauthorized access, dry-run preview, and true failures.

## Standard response envelope

Every React-facing n8n webhook should return JSON shaped like this:

```
{
  "success": true,
  "workflow": "GDA.api.example",
  "action": "list",
  "dryRun": false,
  "data": [],
  "meta": {
    "count": 0,
    "generatedAt": "2026-04-27T18:00:00.000Z",
    "source": "n8n"
  },
  "error": null
}
```

## Required rules

| Rule       | Requirement                                                                                           |
|------------|-------------------------------------------------------------------------------------------------------|
| JSON only  | Do not return HTML, blank strings, or plain text to React-facing endpoints.                           |
| success    | Boolean. true for a completed request, even if the data array is empty.                               |
| workflow   | Workflow name or short stable identifier.                                                             |
| action     | The action routed inside the workflow, such as list, generate, save, health, dryRun.                  |
| dryRun     | Boolean. Required for any write-capable or external-action workflow.                                  |
| data       | Array or object with the actual usable result. Empty data should be [ ] or { }, not a blank response. |
| meta.count | Count of returned records when applicable.                                                            |
| error      | null on success; structured object on failure.                                                        |

## Standard error envelope

```
{
  "success": false,
  "workflow": "GDA.api.example",
  "action": "list",
  "dryRun": false,
  "data": null,
  "meta": {
    "generatedAt": "2026-04-27T18:00:00.000Z"
  },
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Missing or invalid x-gda-key",
    "node": "Auth Guard",
    "retryable": false
  }
}
```

## HTTP status guidance

| Situation                     | HTTP status | success | Notes                                                                    |
|-------------------------------|-------------|---------|--------------------------------------------------------------------------|
| Normal success with data      | 200         | true    | Preferred happy path.                                                    |
| Normal success but empty data | 200         | true    | Empty is not automatically failure. Include <code>meta.count: 0</code> . |
| Dry-run preview accepted      | 200         | true    | Must include <code>dryRun: true</code> .                                 |
| Accepted long-running job     | 202         | true    | Include job id or message.                                               |
| Bad request/body missing      | 400         | false   | Include exact missing field.                                             |
| Unauthorized                  | 401/403     | false   | Do not throw raw stack to React.                                         |
| Internal workflow error       | 500         | false   | Include safe node name/message, not secrets.                             |

## Dry-run standard

Any workflow that can write, delete, send, spend credits, or make irreversible external calls must support one of these before it joins Phase B automation:

```
{
  "dryRun": true
}
```

or

```
{
  "mode": "preview"
}
```

Dry-run must return:

```
{
  "success": true,
  "dryRun": true,
  "action": "create",
  "data": {
    "wouldWrite": true,
    "targetTable": "risk_register",
    "preview": {}
  }
}
```

Dry-run must not:

- Insert production rows
- Update production rows
- Delete production rows
- Send Telegram/email/messages
- Call paid APIs unless explicitly named as a preview-cost test and approved
- Trigger deploys or bulk ingest

### Priority workflows to standardize first

| Priority | Workflow                     | Why                                                                        |
|----------|------------------------------|----------------------------------------------------------------------------|
| 1        | GDA.api.platform-health      | QA Center baseline depends on it.                                          |
| 2        | GDA.api.dashboard-mega       | Launchpad executive dashboard depends on it.                               |
| 3        | GDA.api.opp-tracker 2        | Opportunity list/search depends on it.                                     |
| 4        | GDA.api.capture-plan         | High-value workflow with AI calls and save path; needs preview envelope.   |
| 5        | GDA.api.risk-intel           | Dry-run create already works; standardize all actions.                     |
| 6        | GDA.api.save-opp             | Dry-run already works; standardize response shape.                         |
| 7        | GDA.api.semantic-search      | Search should never return { } without explanation.                        |
| 8        | GDA.research.deep-research   | Requires preview/approval because it touches paid/external APIs.           |
| 9        | GDA.cron.competitor-crawler  | Requires preview/approval because it touches external search and Telegram. |
| 10       | GDA.api.competitor-watchlist | Requires preview/approval because it touches AI/search tools and writes.   |

## React QA Center interpretation

The React QA Center should treat these as red flags:

- HTTP 401 or 403: auth/proxy/key problem
- HTTP 500: workflow logic or database problem
- Empty body: response node problem
- HTML body: proxy or route problem
- `success: false`: workflow-level failure
- Missing `dryRun: true` from a dry-run endpoint: unsafe automation problem

## Bottom line

This standard is the bridge between n8n and React. Without it, every page has custom guessing logic. With it, React can show a plain-English status and the controlled fix agent can patch workflows safely with human approval.